

Assignment no : 04

Roll No : 13216

Problem Statement : Data Analytics I Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

x=np.array([95,85,80,70,60])
y=np.array([85,95,70,65,70])

model= np.polyfit(x, y, 1)

model

array([ 0.64383562, 26.78082192])

predict = np.polyld(model)
predict(65)

np.float64(68.63013698630135)

y_pred= predict(x)
y_pred

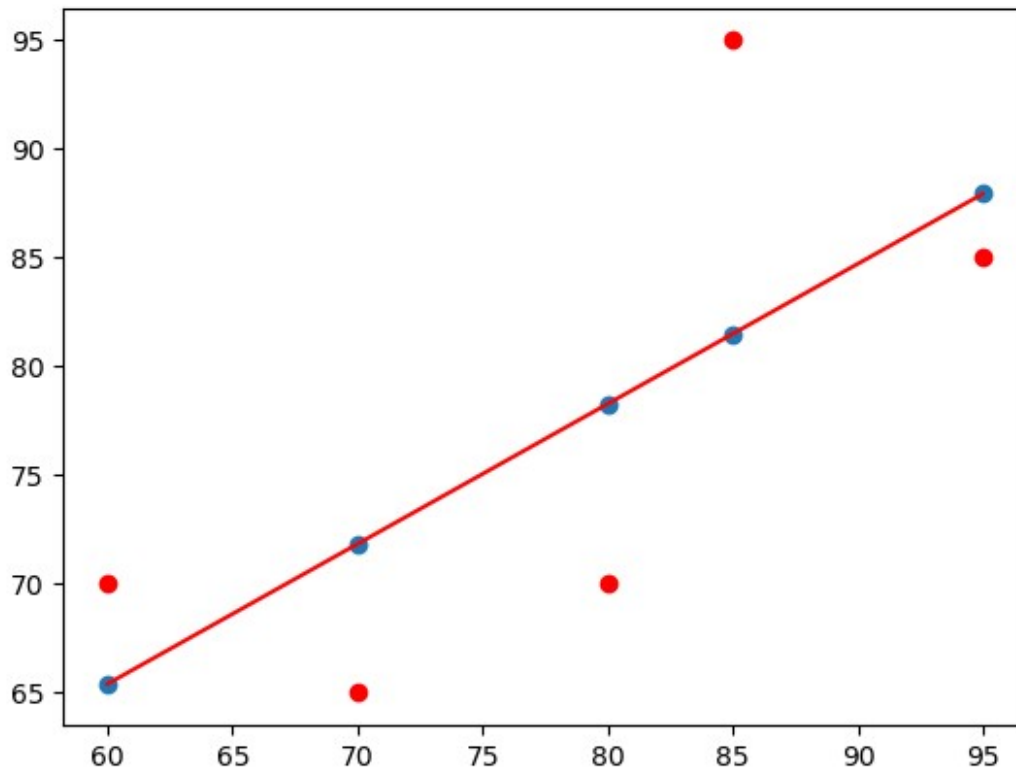
array([87.94520548, 81.50684932, 78.28767123, 71.84931507,
65.4109589 ])

from sklearn.metrics import r2_score
r2_score(y, y_pred)

0.4803218090889323

y_line = model[1] + model[0]* x
plt.plot(x, y_line, c = 'r')
plt.scatter(x, y_pred)
plt.scatter(x,y,c='r')

<matplotlib.collections.PathCollection at 0x2d35b5a6210>
```



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("HousingData.csv")
df.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222

	B	LSTAT	MEDV
0	396.90	4.98	24.0
1	396.90	9.14	21.6
2	392.83	4.03	34.7
3	394.63	2.94	33.4
4	396.90	NaN	36.2

```
df.shape
```

(506, 14)

```
X = df.drop("MEDV",axis=1)
```

```
Y=df["MEDV"]
```

```
feature_names=X.columns
```

boston={

```
"data":X,
```

```
"target":Y,
```

```
"feature_names": feature_names
```

}

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 506 entries, 0 to 505
```

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	CRIM	486 non-null	float64
1	ZN	486 non-null	float64
2	INDUS	486 non-null	float64
3	CHAS	486 non-null	float64
4	NOX	506 non-null	float64
5	RM	506 non-null	float64
6	AGE	486 non-null	float64
7	DIS	506 non-null	float64
8	RAD	506 non-null	int64
9	TAX	506 non-null	int64
10	PTRATIO	506 non-null	float64
11	B	506 non-null	float64
12	LSTAT	486 non-null	float64
13	MEDV	506 non-null	float64

```
dtypes: float64(12), int64(2)
```

```
memory usage: 55.5 KB
```

```
data=pd.DataFrame(df)
```

```
data.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX
0 PTRATIO \ 15.3	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296
1 17.8	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242
2 17.8	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242
3 18.7	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222

```
4 0.06905 0.0 2.18 0.0 0.458 7.147 54.2 6.0622 3 222
18.7
```

```
      B  LSTAT  MEDV
0 396.90  4.98  24.0
1 396.90  9.14  21.6
2 392.83  4.03  34.7
3 394.63  2.94  33.4
4 396.90   NaN  36.2
```

```
data.columns
```

```
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS',
      'RAD', 'TAX',
      'PTRATIO', 'B', 'LSTAT', 'MEDV'],
      dtype='object')
```

```
data=X.copy()
data.columns=X.columns
data.head()
```

```
      CRIM    ZN  INDUS  CHAS    NOX    RM  AGE    DIS  RAD  TAX
PTRATIO \
0 0.00632 18.0   2.31   0.0  0.538  6.575  65.2  4.0900   1  296
15.3
1 0.02731  0.0   7.07   0.0  0.469  6.421  78.9  4.9671   2  242
17.8
2 0.02729  0.0   7.07   0.0  0.469  7.185  61.1  4.9671   2  242
17.8
3 0.03237  0.0   2.18   0.0  0.458  6.998  45.8  6.0622   3  222
18.7
4 0.06905  0.0   2.18   0.0  0.458  7.147  54.2  6.0622   3  222
18.7
```

```
      B  LSTAT
0 396.90  4.98
1 396.90  9.14
2 392.83  4.03
3 394.63  2.94
4 396.90   NaN
```

```
data.rename(columns={'medv':'PRICE'},inplace=True)
data.head()
```

```
      CRIM    ZN  INDUS  CHAS    NOX    RM  AGE    DIS  RAD  TAX
PTRATIO \
0 0.00632 18.0   2.31   0.0  0.538  6.575  65.2  4.0900   1  296
15.3
1 0.02731  0.0   7.07   0.0  0.469  6.421  78.9  4.9671   2  242
17.8
2 0.02729  0.0   7.07   0.0  0.469  7.185  61.1  4.9671   2  242
```

```

17.8
3  0.03237    0.0    2.18    0.0    0.458    6.998    45.8    6.0622    3    222
18.7
4  0.06905    0.0    2.18    0.0    0.458    7.147    54.2    6.0622    3    222
18.7

```

```

      B  LSTAT
0  396.90    4.98
1  396.90    9.14
2  392.83    4.03
3  394.63    2.94
4  396.90    NaN

```

```
data.isnull().sum()[data.isnull().sum(>0)]
```

```

CRIM      20
ZN        20
INDUS     20
CHAS      20
AGE       20
LSTAT     20
dtype: int64

```

```
data.columns=data.columns.str.strip()
```

```
target_columns=data.columns[-1]
```

```
X=data.drop(target_columns,axis=1)
```

```
Y=data[target_columns]
```

```
print("Target Columns is : ",target_columns)
```

```
Target Columns is :  LSTAT
```

```
print(X.shape, Y.shape)
```

```
(506, 12) (506,)
```

```

from sklearn.model_selection import train_test_split
xtrain, xtest, ytrain, ytest=train_test_split(
    X,Y,
    test_size=0.2,
    random_state=0
)

```

```
print(xtrain.shape)
```

```
(404, 12)
```

```
print(xtest.shape)
```

```
(102, 12)
```

```

print(ytrain.shape)
(404,)
print(ytest.shape)
(102,)
from sklearn.linear_model import LinearRegression
lm=LinearRegression()

print(xtrain.shape)
print(ytrain.shape)
(404, 12)
(404,)
print(xtrain.dtypes)
CRIM      float64
ZN        float64
INDUS     float64
CHAS      float64
NOX       float64
RM        float64
AGE       float64
DIS       float64
RAD       int64
TAX       int64
PTRATIO   float64
B         float64
dtype: object
print(xtrain.select_dtypes(include=['object']).columns)
Index([], dtype='object')
xtrain = pd.get_dummies(xtrain)

import numpy as np
print("NaN in xtrain:", np.isnan(xtrain).sum().sum())
print("NaN in ytrain:", np.isnan(ytrain).sum())
NaN in xtrain: 78
NaN in ytrain: 19
print("Inf in xtrain:", np.isinf(xtrain).sum().sum())

```

```
Inf in xtrain: 0
```

```
print(xtrain[:5])    # instead of head()
print(xtrain.shape)
print(type(xtrain))
```

```
[[3.58090000e-01 0.00000000e+00 6.20000000e+00 1.00000000e+00
  5.07000000e-01 6.95100000e+00 8.85000000e+01 2.86170000e+00
  8.00000000e+00 3.07000000e+02 1.74000000e+01 3.91700000e+02]
 [1.58760000e-01 0.00000000e+00 1.08100000e+01 0.00000000e+00
  4.13000000e-01 5.96100000e+00 1.75000000e+01 5.28730000e+00
  4.00000000e+00 3.05000000e+02 1.92000000e+01 3.76940000e+02]
 [1.13290000e-01 3.00000000e+01 4.93000000e+00 6.97674419e-02
  4.28000000e-01 6.89700000e+00 5.43000000e+01 6.33610000e+00
  6.00000000e+00 3.00000000e+02 1.66000000e+01 3.91250000e+02]
 [8.82900000e-02 1.25000000e+01 7.87000000e+00 6.97674419e-02
  5.24000000e-01 6.01200000e+00 6.66000000e+01 5.56050000e+00
  5.00000000e+00 3.11000000e+02 1.52000000e+01 3.95600000e+02]
 [2.59406000e+01 0.00000000e+00 1.81000000e+01 0.00000000e+00
  6.79000000e-01 5.30400000e+00 8.91000000e+01 1.64750000e+00
  2.40000000e+01 6.66000000e+02 2.02000000e+01 1.27360000e+02]]
(404, 12)
<class 'numpy.ndarray'>
```

```
print(type(ytrain))
print(ytrain.shape)
print(ytrain[:10])
```

```
<class 'pandas.core.series.Series'>
(404,)
220    9.71
71     9.88
240   11.38
6     12.43
417   26.64
366   14.00
502    9.08
302    8.67
157    4.59
408   26.40
Name: LSTAT, dtype: float64
```

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.impute import SimpleImputer
```

```
# Step 1: Ensure xtrain is DataFrame
xtrain = pd.DataFrame(xtrain)
```

```

# Step 2: Convert categorical → numeric
xtrain = pd.get_dummies(xtrain)

# Step 3: Replace inf with NaN
xtrain = xtrain.replace([np.inf, -np.inf], np.nan)

# Step 4: Handle missing values
imputer = SimpleImputer(strategy="mean")
xtrain = imputer.fit_transform(xtrain)

# Step 5: Fix ytrain
ytrain = np.nan_to_num(ytrain)

# Step 6: Train model
lm = LinearRegression()
model = lm.fit(xtrain, ytrain)

print("Model training completed")
Model training completed

from sklearn.metrics import r2_score

ypred = model.predict(xtrain)

print("R2 Score:", r2_score(ytrain, ypred))
R2 Score: 0.6120438995872886

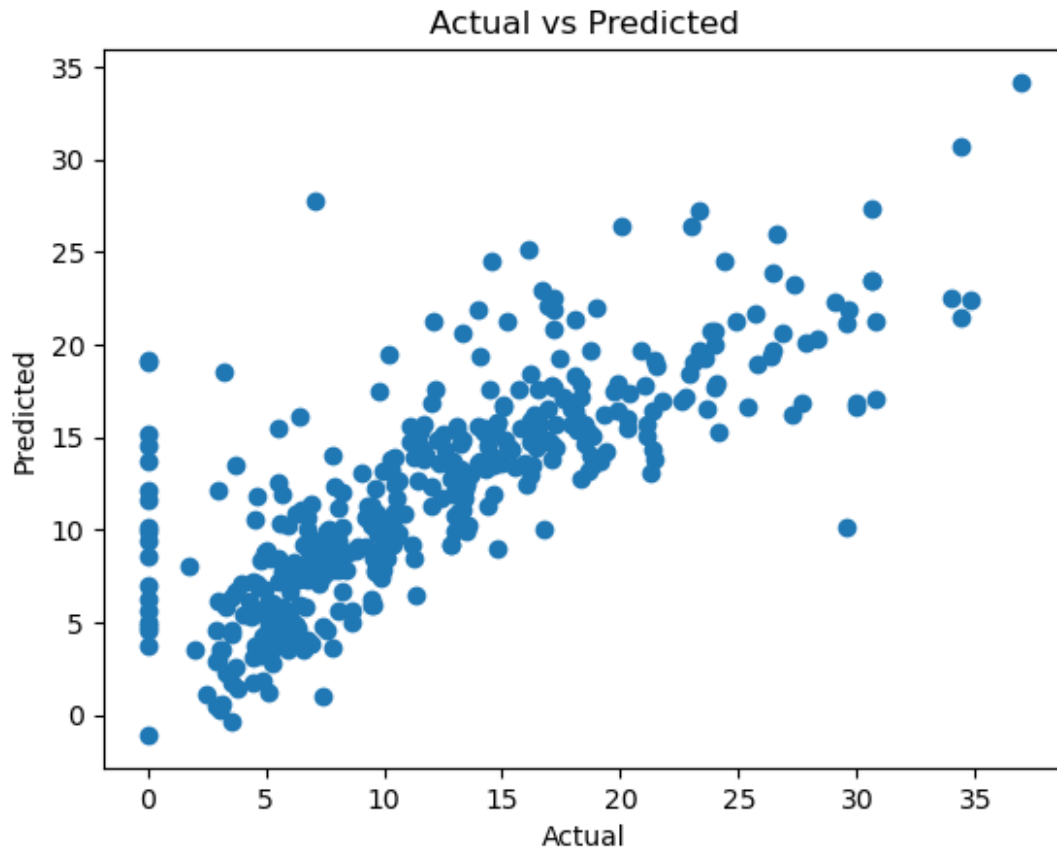
print(model.coef_)
print(model.intercept_)

[ 1.31274686e-01  2.92663516e-02  6.50059341e-02  7.63857200e-01
  4.05795112e+00 -4.79734785e+00  9.59730766e-02 -1.22154319e-02
 -1.97611160e-02 -1.18783335e-03  1.21191615e-01 -1.71833248e-02]
36.603530404238555

import matplotlib.pyplot as plt

plt.scatter(ytrain, ypred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.show()

```

```
from sklearn.linear_model import LinearRegression

lm = LinearRegression()
lm.fit(xtrain, ytrain)

LinearRegression()

xtest = pd.get_dummies(xtest)

import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression

# Convert to DataFrame
xtrain = pd.DataFrame(xtrain)
xtest = pd.DataFrame(xtest)

# Apply get_dummies
xtrain = pd.get_dummies(xtrain)
xtest = pd.get_dummies(xtest)

# Save columns
xtrain_columns = xtrain.columns
```

```
# Align xtest to xtrain
xtest = xtest.reindex(columns=xtrain_columns, fill_value=0)
```

```
# Replace inf
xtrain = xtrain.replace([np.inf, -np.inf], np.nan)
xtest = xtest.replace([np.inf, -np.inf], np.nan)
```

```
# Impute
imputer = SimpleImputer(strategy="mean")
xtrain = imputer.fit_transform(xtrain)
xtest = imputer.transform(xtest)
```

```
# Train
lm = LinearRegression()
lm.fit(xtrain, ytrain)
```

```
# Predict
ytrain_pred = lm.predict(xtrain)
ytest_pred = lm.predict(xtest)
```

```
print("All good ☑")
```

All good ☑

```
from sklearn.metrics import r2_score
```

```
# Predictions
ytrain_pred = lm.predict(xtrain)
ytest_pred = lm.predict(xtest)
```

```
# Fix shape if needed
ytest = ytest.ravel()
```

```
# Scores
print("Train R2:", r2_score(ytrain, ytrain_pred))
print("Test R2:", r2_score(ytest, ytest_pred))
```

Train R2: 0.6120438995872886

```
C:\Users\Kiran Janjal\AppData\Local\Temp\
ipykernel_8532\3624623420.py:8: FutureWarning: Series.ravel is
deprecated. The underlying array is already 1D, so ravel is not
necessary. Use `to_numpy()` for conversion to a numpy array instead.
    ytest = ytest.ravel()
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
Cell In[66], line 12
```

```

10 # Scores
11 print("Train R2:", r2_score(ytrain, ytrain_pred))
--> 12 print("Test R2:", r2_score(ytest, ytest_pred))

File ~\anaconda3\Lib\site-packages\sklearn\utils\
_param_validation.py:218, in
validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
    212 try:
    213     with config_context(
    214         skip_parameter_validation=(
    215             prefer_skip_nested_validation or
global_skip_validation
    216         )
    217     ):
--> 218     return func(*args, **kwargs)
    219 except InvalidParameterError as e:
    220     # When the function is just a wrapper around an estimator,
we allow
    221     # the function to delegate validation to the estimator,
but we replace
    222     # the name of the estimator by the name of the function in
the error
    223     # message to avoid confusion.
    224     msg = re.sub(
    225         r"parameter of \w+ must be",
    226         f"parameter of {func.__qualname__} must be",
    227         str(e),
    228     )

File ~\anaconda3\Lib\site-packages\sklearn\metrics\
_regression.py:1276, in r2_score(y_true, y_pred, sample_weight,
multioutput, force_finite)
    1152 """ $R^2$  (coefficient of determination) regression score
function.
    1153
    1154 Best possible score is 1.0 and it can be negative (because the
(...) 1269 -inf
    1270 """
    1271 xp, _, device_ = get_namespace_and_device(
    1272     y_true, y_pred, sample_weight, multioutput
    1273 )
    1275 _, y_true, y_pred, sample_weight, multioutput = (
-> 1276     _check_reg_targets_with_floating_dtype(
    1277         y_true, y_pred, sample_weight, multioutput, xp=xp
    1278     )
    1279 )
    1281 if _num_samples(y_pred) < 2:
    1282     msg = "R^2 score is not well-defined with less than two
samples."

```

```
File ~\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:209,
in _check_reg_targets_with_floating_dtype(y_true, y_pred,
sample_weight, multioutput, xp)
```

```
    160 """Ensures y_true, y_pred, and sample_weight correspond to
same regression task.
```

```
    161
```

```
    162 Extends `_check_reg_targets` by automatically selecting a
suitable floating-point
```

```
    (...)    205     correct keyword.
```

```
    206 """
```

```
    207 dtype_name = _find_matching_floating_dtype(y_true, y_pred,
sample_weight, xp=xp)
```

```
--> 209 y_type, y_true, y_pred, sample_weight, multioutput =
_check_reg_targets(
```

```
    210     y_true, y_pred, sample_weight, multioutput,
dtype=dtype_name, xp=xp
```

```
    211 )
```

```
    213 return y_type, y_true, y_pred, sample_weight, multioutput
```

```
File ~\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:115,
in _check_reg_targets(y_true, y_pred, sample_weight, multioutput,
dtype, xp)
```

```
    112 xp, _ = get_namespace(y_true, y_pred, multioutput, xp=xp)
```

```
    114 check_consistent_length(y_true, y_pred, sample_weight)
```

```
--> 115 y_true = check_array(y_true, ensure_2d=False, dtype=dtype)
```

```
    116 y_pred = check_array(y_pred, ensure_2d=False, dtype=dtype)
```

```
    117 if sample_weight is not None:
```

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1105,
in check_array(array, accept_sparse, accept_large_sparse, dtype,
order, copy, force_writeable, force_all_finite, ensure_all_finite,
ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples,
ensure_min_features, estimator, input_name)
```

```
    1099     raise ValueError(
```

```
    1100         f"Found array with dim {array.ndim},"
```

```
    1101         f" while dim <= 2 is required{context}."
```

```
    1102     )
```

```
    1104 if ensure_all_finite:
```

```
-> 1105     _assert_all_finite(
```

```
    1106         array,
```

```
    1107         input_name=input_name,
```

```
    1108         estimator_name=estimator_name,
```

```
    1109         allow_nan=ensure_all_finite == "allow-nan",
```

```
    1110     )
```

```
    1112 if copy:
```

```
    1113     if _is_numpy_namespace(xp):
```

```
    1114         # only make a copy if `array` and `array_orig` may
share memory`
```

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:120, in
```

```

_assert_all_finite(X, allow_nan, msg_dtype, estimator_name,
input_name)
    117 if first_pass_isfinite:
    118     return
--> 120 _assert_all_finite_element_wise(
    121     X,
    122     xp=xp,
    123     allow_nan=allow_nan,
    124     msg_dtype=msg_dtype,
    125     estimator_name=estimator_name,
    126     input_name=input_name,
    127 )

```

File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:169, in _assert_all_finite_element_wise(X, xp, allow_nan, msg_dtype, estimator_name, input_name)

```

    152 if estimator_name and input_name == "X" and has_nan_error:
    153     # Improve the error message on how to handle missing
values in
    154     # scikit-learn.
    155     msg_err += (
    156         f"\n{estimator_name} does not accept missing values"
    157         " encoded as NaN natively. For supervised learning,
you might want"
    (... ) 167         "#estimators-that-handle-nan-values"
    168     )
--> 169 raise ValueError(msg_err)

```

ValueError: Input contains NaN.